Team Presentation Guide

When should Night Ride move a waiting request to Lyft?

> **THE STORY IN ONE PARAGRAPH**
>
> We wanted to study dispatch decisions. April's data described aggregate patterns rather than individual journeys, so we built plausible evenings from evidence and explicit assumptions. We checked the implementation, replayed identical evenings under different transfer rules, and found a conditional trade-off between timely service and Lyft use. Our next work tests how robust those findings are.

Pack version	Team share v1.0 (23 October 2026)
Presentation	Final slides (built as content draft v0.4): 28 main slides, backup divider on slide 29, backups on slides 30–38
Methodology	v1.6 (file 03_Scenario_Methodology_v1.6.pdf)
Timing	About 26:26 of narration plus about 45 s of handoffs (about 27 minutes); **an estimate, not yet rehearsed**

How to read: blue boxes are the exact words to say; grey boxes are preparation material that is not spoken. Every slide title in the contents is a link, and every slide section links to the deck and the methodology PDF in this folder.

Contents

1 Presenters and section boundaries 4

2 Quick Q&A navigation 6

3 Plain-language reference 9

4 What every member must be able to explain 11

5 Human verification activities 11

6 Slide by slide 13

1 · When should Night Ride move a waiting request to Lyft? . . . 14

2 · The service and one request . . . 15

3 · Where we started . . . 16

4 · Where we started: numbers we later corrected . . . 17

5 · We narrowed the question to one controllable decision . . . 18

6 · April describes totals—not individual journeys . . . 20

7 · How busy should a constructed evening be? . . . 21

8 · We started with a smaller, deliberately demanding hour . . . 23

9 · Knowing the endpoints does not identify the journeys . . . 24

10 · One average can hide very different groups . . . 25

11 · One status label hides different events . . . 26

12 · The simulator needs a probability at vehicle arrival . . . 27

13 · The evidence becomes a scenario blueprint . . . 28

14 · Three components have different jobs . . . 29

15 · The same decision cycle repeats as the evening unfolds . . . 30

16 · The optimizer compares the consequences of candidate plans . . . 32

17 · We changed the transfer decision, not the van dispatcher . . . 34

18 · What does ‘Lyft arrives sooner’ actually compare? . . . 36

19 · A cheap postponement price could keep a rider waiting . . . 37

20 · Replay the same night; change one rule . . . 39

21 · We checked implementation before interpreting outcomes . . . 40

22 · The predictive rules reached more requests promptly—and used more Lyft . . . 42

23 · Moving a request to Lyft does not end its wait . . . 44

24 · Better service and lower Lyft use can point to different choices 46
25 · Could we choose the transfer policy systematically? 48
26 · Which assumptions could change the answer? 50
27 · Test robustness before fine-tuning a winner 51
28 · What we can defend today 52

1 Presenters and section boundaries

Presenter	Slides	From → to	Responsibility	Time
Irfan Setiadi	1–5	When should Night Ride move a waiting request to Lyft? → We narrowed the question to one controllable decision	Introduction, service context, proposal recap and refined research question	4:13
Miles Zhou	6–13	April describes totals—not individual journeys → The evidence becomes a scenario blueprint	Aggregate evidence, workload, scenario construction and assumptions	7:47
Youzhi Zhang	14–21	Three components have different jobs → We checked implementation before interpreting outcomes	Architecture, decision cycle, mathematics, transfer rules, worked example, postponement issue and verification	8:10
Vignesh Rajesh Babu	22–28	The predictive rules reached more requests promptly—and used more Lyft → What we can defend today	Findings, transfer outcomes, Pareto trade-offs, leader–follower extension, limitations, next steps and conclusion	6:16

Irfan Setiadi opens the talk and is also the design lead for the slides. Times are spoken narration plus about 4 s of pause per slide, from the finalized script: **about 26:26 in total plus about 45 s for three handoffs, about 27 minutes**. Reading numbers slowly can add about a minute; dropping the [optional] sentences saves about 1.5 minutes. This is an estimate until the team has timed a full rehearsal; the limit is 30 minutes.

Longer technical sections. Miles (8 slides) and Youzhi (8 slides) carry the longest sections. Rehearse them with a stopwatch per slide: slides 14–16 (architecture, decision cycle, mathematics) and 7 (workload) are the densest. Sentences marked [optional] in the narration can be dropped if a section runs long; do not drop the qualifications in bold on the slides.

Exact handoff sentences

- **Irfan Setiadi → Miles Zhou** (end of slide 5): “That gives us the question. Miles will now explain how April’s aggregate data became the ingredients of our scenarios.”
- **Miles Zhou → Youzhi Zhang** (end of slide 13): “Those inputs define the constructed evening. Youzhi will explain how the model makes decisions as that evening unfolds.”
- **Youzhi Zhang → Vignesh Rajesh Babu** (end of slide 21): “We then replayed the same evenings under different transfer rules. Vignesh will explain what changed, what it means and what remains uncertain.”

The incoming presenter starts directly with the first sentence of the next slide, with no re-introduction.

Every main slide and its presenter

#	Title	Presenter	s
1	When should Night Ride move a waiting request to Lyft?	Irfan	42
2	The service and one request	Irfan	55
3	Where we started	Irfan	52
4	Where we started: numbers we later corrected	Irfan	50
5	We narrowed the question to one controllable decision	Irfan	54
6	April describes totals—not individual journeys	Miles	58
7	How busy should a constructed evening be?	Miles	64
8	We started with a smaller, deliberately demanding hour	Miles	56
9	Knowing the endpoints does not identify the journeys	Miles	49
10	One average can hide very different groups	Miles	61
11	One status label hides different events	Miles	62
12	The simulator needs a probability at vehicle arrival	Miles	55
13	The evidence becomes a scenario blueprint	Miles	62
14	Three components have different jobs	Youzhi	65
15	The same decision cycle repeats as the evening unfolds	Youzhi	58
16	The optimizer compares the consequences of candidate plans	Youzhi	76
17	We changed the transfer decision, not the van dispatcher	Youzhi	69
18	What does ‘Lyft arrives sooner’ actually compare?	Youzhi	64
19	A cheap postponement price could keep a rider waiting	Youzhi	60
20	Replay the same night; change one rule	Youzhi	44
21	We checked implementation before interpreting outcomes	Youzhi	54
22	The predictive rules reached more requests promptly—and used more Lyft	Vignesh	64
23	Moving a request to Lyft does not end its wait	Vignesh	58
24	Better service and lower Lyft use can point to different choices	Vignesh	66
25	Could we choose the transfer policy systematically?	Vignesh	66
26	Which assumptions could change the answer?	Vignesh	54
27	Test robustness before fine-tuning a winner	Vignesh	40
28	What we can defend today	Vignesh	28

2 Quick Q&A navigation

Route each question to its owner rather than improvising. The guide column links to the slide's preparation notes; the backup column opens the deck at that page (in slide-show mode, type the number and press Enter).

Question is about	Owner	Guide (slide)	Backup slide
Why this project; what the service is	Irfan	1, 2	—
The original proposal and what changed	Irfan	3, 4	38
Why hold one dispatcher fixed (the pivot)	Irfan	5	38
What April's data do and do not contain	Miles	6	33
Where 168 requests and 12.85 per vehicle come from	Miles	7	33
The two-van pilot and the burst	Miles	8	33, 35
Trip pairing, party sizes, patience, no-shows	Miles	9, 10, 11, 12	33
What is observed, derived or assumed	Miles	6, 13	33
Simulator, optimizer and rule: who does what	Youzhi	14, 15	32
The objective, 'optimal', candidate restriction	Youzhi	16	31, 37
Exact rule definitions and eligibility	Youzhi	17	30
The worked transfer timeline	Youzhi	18	—
The postponement-price flaw and fix	Youzhi	19	34
Paired replay; 30 nights, not 180	Youzhi	20	36
Verification vs validation; AI-written code	Youzhi	21	37
Results and the 15-minute measure	Vignesh	22	35
Losses after transfer; patience interpretation	Vignesh	23	35
Pareto dominance and its limits	Vignesh	24	36
Leader–follower / bilevel extension	Vignesh	25	31
Limits, next experiments, the conditional answer	Vignesh	26, 27, 28	37

Backup slides

Slide	Code	Content
30	B1	Code key: plain rule names and file codes
31	B2	The route-selection MILP and the M bound
32	B3	Event order, triggers and commitments
33	B4	Where each input comes from
34	B5	Three postponement prices compared
35	B6	All metrics by rule, 30 nights
36	B7	Trade-off night by night
37	B8	Numerical diagnostics
38	B9	Corrected proposal claims

Every likely question in this guide

Each question links to the slide section that answers it.

Question	Slide	Owner
Is this a real operational question?	1	Irfan
Why “booking” and “Lyft order”?	2	Irfan
Can a rider refuse the Lyft?	2	Irfan
Did you ask for the trip data?	3	Irfan
Why not compare dispatch algorithms, as proposed?	5	Irfan
Is your dispatcher Hopkins’ dispatcher?	5	Irfan
Why per vehicle?	7	Miles
Why does hour 3 (54) exceed hour 2 (50)?	7	Miles
Why two vans, not five?	8	Miles
Is 14 seats realistic?	8	Miles
Why prefer nearby trips?	9	Miles
Why not fit the distribution?	10	Miles
Why 18 minutes?	11	Miles
Your scenario files say 4.7%?	12	Miles
Is a no-show related to how long the rider waited?	12	Miles
Does the rule see the hidden traits?	14	Youzhi
Could transfers cascade?	15	Youzhi
Why at most 3 new pickups?	16	Youzhi
Optimal, given 41 incomplete tie searches?	16	Youzhi
Did the transfer help this rider?	18	Youzhi
Your fix made the median worse?	19	Youzhi
Who paid for the lower worst wait?	19	Youzhi
Why “the one that removes the flaw” rather than the simplest?	19	Youzhi
Are 30 nights enough?	20	Youzhi
Who wrote the code and checks?	21	Youzhi
Isn’t your metric rigged for early transfer?	22	Vignesh

Question	Slide	Owner
Your predictive rules transfer at 0 s elapsed. Isn't that just sending the burst straight to Lyft?	22	Vignesh
Do the pilot nights resemble April?	22	Vignesh
Is the prediction rule better?	22	Vignesh
Why did "Unassigned for ten minutes" do almost nothing?	22	Vignesh
Isn't this just your patience assumption?	23	Vignesh
Per night, are fewer riders lost after transfer under the prediction rules?	23	Vignesh
Is the Pareto set robust to noise?	24	Vignesh
Why Lyft orders, not cost?	24	Vignesh
Is this the Pareto frontier?	24	Vignesh
Isn't your loop already leader–follower?	25	Vignesh
Riders and Lyft as players?	25	Vignesh
Why bilevel and not just simulation optimisation?	25	Vignesh
Is your follower optimistic or pessimistic?	25	Vignesh
So which rule should Hopkins use?	28	Vignesh

3 Plain-language reference

Term	Meaning in this project
Request (booking)	One Night Ride request, possibly with several passengers. Files call it a booking.
Lyft order	A separate order placed after a transfer decision (files: “booking”). Placed 2 min after the decision (the ordering lag).
Curb arrival	A vehicle reaches the pickup location. Waiting time runs from the request to curb arrival.
Pickup	The rider boards. “Reached within 15 min” uses curb arrival, not boarding.
Cancellation	The rider gives up before a vehicle reaches the curb (also possible after a transfer, before the Lyft arrives).
No-show	The rider is absent when the vehicle arrives; the van waits 3 min.
Constructed night	One synthetic one-hour pilot night: 28 requests, 2 vans of 14 seats, 11 requests placed in an 18:20–18:30 burst.
Paired replay	The same night replayed under each rule with identical requests and hidden traits: 30 nights $\times$ 6 rules = 30 paired scenarios, not 180 independent nights.
Simulator / van optimizer / transfer rule	Advances time and reveals events / chooses among feasible candidate plans (a mixed-integer linear program, MILP) / decides whether an eligible request moves to Lyft.
Decision moment (epoch)	When the dispatcher re-plans: every 2 min, at each new request, at a cancellation or no-show while waiting for a van, and when a van comes on duty.
Postponement price	What the optimizer charges for leaving a request unserved this round; a large constant makes it serve as many requests as its candidates allow.
Verification / validation	The code implements the specified model / the model represents the real service. Only verification has been done.
Dominated (Pareto)	Another tested rule is at least as good on both objectives and strictly better on one. Nondominated is not recommended.

The six transfer rules (codes for files only)

Rule	Moves a waiting request to Lyft when	Code
Vans only	never	R0
Wait ten / fifteen minutes	it has waited at least 10 / 15 minutes	R1(10) / R1(15)
Unassigned for ten minutes	it has been outside every van plan for 10 minutes in a row	R1u(10)
Predicted total wait exceeds fifteen minutes	predicted pickup clock time $-$ request clock time $>$ 15 min	R2(15)
Lyft arrives sooner	the predicted van pickup is later than now + 2 (lag) + 8 (typical delay) + 2 (margin) minutes	R3(2)

Labels on the slides: **evidence** (OBSERVED, DERIVED, ASSUMED, DESIGNED, SIMULATED, ILLUSTRATIVE) and **execution status** (TESTED IN SIMULATION, PLANNED — NOT YET RUN, PROPOSED — NOT IMPLEMENTED). “Tested” never means validated against Hopkins.

Key numbers

6 pm completed rides	5,089 digitized (tooltip 5,098), a 30-day total; about 170 per evening
Requests per completed ride; per vehicle-hour	2.087 (50,297 / 24,095); about 12.85 at 6 pm
Planned five-van evening (not run)	64 / 50 / 54 = 168 requests
No-show	4.7% of all requests; 8.9% per van arrival (conditional on the status mapping)
Reached within 15 min (30-night means)	vans only 48%; waiting rules 50–51%; predictive rules 73–75%
Lyft orders per night	0 / 5.9 / 3.1 / 0.2 / 9.3 / 10.5 (rules in the order above)
Lost after transfer	95 of 196 under Wait ten minutes; about a third under the predictive rules

References you will see in the notes

The preparation notes keep some references to the team's repository for teammates who have access. You do not need them for Q&A: use the methodology page references instead.

Reference	What it is, and where to look in this pack
D-xx (for example D-49)	A numbered entry in the project decision log (repository). The methodology summarizes the decision history in Appendix A (p. 47).
spec §N; model design §N	The scenario specification and the model-design document (repository). Their rules are restated in methodology §4 (simulator) and §5 (mathematics).
Technical review A, B, C	The v0.4 review that settled the patience interpretation (A), the reason for the pivot (B) and the leader–follower wording (C). Its conclusions are in the notes for slides 23, 5 and 25.
TransLoc notes §N	Team research notes on the vendor's published dispatch design (repository).
Phase 4 log; L1–L5	The project's build log (repository); L1–L4 are the small designed evenings and L5 the two-van pilot (code key on slide 30).
File paths (data/…, outputs/…, src/…, .py, .md, .csv)	Repository files, for teammates with access only.
Slide source lines	Some deck footers cite "methodology v1.5 §1a" or "§7b"; in the v1.6 PDF these are §1.4 (p. 8) and §7.8–7.9 (pp. 42–43).

4 What every member must be able to explain

1. **Why the data could not reconstruct individual journeys.** April's dashboards give totals and shares (statuses, rides by hour, place counts, percentiles); no record links one rider's request time, place, wait and outcome. So every evening is constructed. (Slides 6, 9.)
2. **What is observed, derived and assumed.** Observed: read from the dashboards (for example 50,297 requests). Derived: computed from observed numbers with a stated formula (12.85 requests per vehicle-hour, 8.9% no-show per arrival). Assumed: our choices (trip pairing, patience, Lyft delay). Designed: experiment settings we chose (two vans, the burst). (Slides 6, 13.)
3. **Why one dispatcher is held fixed.** A scope choice: the question we chose is the transfer rule, the real dispatcher's internals are unknown to us, and a controlled comparison needs everything else fixed. Synthetic data do not forbid comparing dispatch algorithms. (Slide 5.)
4. **Why a transfer does not end waiting.** After a transfer the rider still waits for the 2-minute ordering lag and the Lyft's travel, and their patience clock keeps running from the original request. Riders can cancel before the Lyft arrives. (Slide 23.)
5. **Verification versus validation.** Verification: does the code implement the specified model (hand keys, enumeration, tests, reruns)? Validation: does the model represent the real service? We have done verification, not validation. (Slide 21.)
6. **Why more service with more Lyft is a trade-off, not a universal winner.** The predictive rules reached more requests within 15 minutes but used about 9–10.5 Lyft orders a night. Choosing needs a value on Lyft use, and the 15-minute measure itself favours early transfer. (Slides 22, 24.)
7. **Why the findings do not establish real Hopkins effects.** They come from 30 constructed two-van nights with placeholder rider behaviour and a dispatcher modelled on a published design, verified but not validated. They are conditional statements about our model. (Slides 20, 26, 28.)

5 Human verification activities

> **THE DISTINCTION**
>
> A program can generate and check an answer. Human verification means a teammate independently works through a defined example, records their calculation and compares it with the implementation.

Each presenter has one guided activity in their presenter brief (folder `presenter_briefs`). Every record starts as **Awaiting human completion**. Reference answers produced with AI assistance do not count as a teammate's verification.

Presenter	Activity	Status
Irfan Setiadi	Units and the proposal correction (Slides 3–4; also slide 7)	Awaiting human completion
Miles Zhou	The no-show denominator (Slide 12; also slide 11)	Awaiting human completion
Youzhi Zhang	One decision moment on a small designed evening (Slides 15–16)	Awaiting human completion
Vignesh Rajesh Babu	One transfer timeline (“Lyft arrives sooner”) (Slides 18 and 23; also slide 17)	Awaiting human completion

6 Slide by slide

Each slide below has the exact narration (blue) and the preparation material (grey). At a handoff, the blue box ends with the handoff sentence.

Slide 1 · When should Night Ride move a waiting request to Lyft?

Irfan Setiadi about 42 s · open slide 1

Why it is here. Opens the talk with the decision, before any data or model.

SAY (exact narration)

We began with a dispatch problem. The data led us first to a modelling problem. Good afternoon, we are Team FourSight. Our question is simple to state: when a rider has been waiting for a Night Ride van, when should that request be moved to Lyft instead? [optional] Today we will show what April's aggregate data let us build, how we turned it into constructed evenings, and what our first simulations revealed.

TRANSITION

First, the service itself, and one request.

PREPARATION (not spoken)

Design choice. The talk opens with the decision, not the data, so that every later slide answers "what does this mean for that decision?".

Likely question.

- **Is this a real operational question?** Yes. JHU's service page says a dispatcher sends the Lyft. We study the timing of that decision in a model, not Hopkins' actual practice (TransLoc notes §4.4).

Backup slides: none specific

Methodology PDF: §1.1 The question, p. 6

Slide 2 · The service and one request

Irfan Setiadi about 55 s · open slide 2

Why it is here. Gives the vocabulary that every later number depends on.

SAY (exact narration)

Homewood-area Night Ride pools requests in shared vans. A dispatcher can move waiting riders to Lyft. We are studying when that transfer should happen. Here is one request's journey: request, wait, the vehicle reaches the curb, boarding, drop-off. It breaks if the rider gives up while waiting, a cancellation, or is absent when the van arrives, a no-show after a three-minute wait. Four words matter later, and they are on the tiles: booking, Lyft order, curb arrival, and pickup, the rider boarding. Each Lyft order is an extra trip Hopkins pays for, so moving riders to Lyft has a price.

TRANSITION

Our September proposal approached this differently.

PREPARATION (not spoken)

Evidence.

- **Scope:** Homewood Night Ride, including the Peabody area (D-27); East Baltimore is excluded (D-23).
- **Curb wait:** 3 minutes (published policy).

Definitions used throughout.

- **Waiting time:** from the request to curb arrival.
- **Patience:** a rider's patience clock runs from the request until a vehicle reaches the curb, *including after a transfer*.
- **Acceptance:** in the pilot every transfer offer is accepted (a simplification); the designed evenings use 90% acceptance.

Why Lyft use counts as a cost. Each Lyft order is an extra trip Hopkins pays for. A rider's app showing no charge only says that Hopkins pays; the cost per trip is unknown to us, so we count orders, not dollars (D-45).

Likely questions.

- **Why "booking" and "Lyft order"?** "Booking" means the rider's Night Ride request; the separate Lyft order is a different object. The files call the Lyft order a "booking" (D-43).
- **Can a rider refuse the Lyft?** In reality yes. The pilot assumes acceptance; the designed evenings include refusals, and a request that declines is not offered again.

Backup slides: none specific

Methodology PDF: §1.3 System boundaries, p. 6; §4.6 Transfers and the Lyft service, p. 22

Slide 3 · Where we started

Irfan Setiadi about 52 s · open slide 3

Why it is here. The mandatory recap, shown with the proposal's own slides.

SAY (exact narration)

Our original plan was to reconstruct service nights and study dispatch decisions using an optimization model. These are two slides from that proposal. On the left, the data plan: aggregate dashboards were available, and de-identified trip-level extracts were coming, with request, pickup and drop-off times for each ride. Trip-level data would let us replay actual service nights. On the right, the method: compare fast heuristics, a rolling-horizon optimization model and a hybrid, all on the same request stream. Those trip-level extracts never arrived, so real nights could not be replayed.

TRANSITION

The proposal also contained numbers we later had to correct.

PREPARATION (not spoken)

Evidence.

- **Source:** proposal slides 10 and 11, from the committed proposal deck (`git show HEAD:docs/presentations/BlueJayNightRide_Proposal.pptx`), reproduced as images.
- **Untouched:** the working copy has an unrelated uncommitted change.
- **Masked:** the dashboard map images on slide 10 are covered, because April dashboard imagery is restricted.

The proposal's actual ambitions (verified against its text).

- Replay actual service nights from de-identified trip-level extracts: timestamps, zones, party size, vehicle IDs (slide 10: "Trip-level data will allow us to replay actual service nights").
- Compare heuristics, a rolling-horizon MILP and a hybrid on the same request stream (slide 11).
- Use the published targets as constraints (slide 6).
- Treat Lyft as paid overflow inside the optimizer (slide 12).

Likely question.

- **Did you ask for the trip data?** Yes. The proposal lists the extracts as "requested". They never arrived, which is why every evening here is constructed.

Backup slides: slide 38 (B9)

Methodology PDF: §1.4 From the September proposal to the current study, p. 8

Slide 4 · Where we started: numbers we later corrected

Irfan Setiadi about 50 s · open slide 4

Why it is here. Corrects superseded numbers instead of silently repeating them.

SAY (exact narration)

Four numbers on these original slides were wrong or misread. One: “about 5,100 rides per hour at 6 pm” is a total over thirty days, so about 170 rides in one evening’s six o’clock hour. Two: “about 28 vans” is the dashboard’s all-services average, not a Homewood-only count. [optional] Three: the 7 pm bar shows 4,900; our digitized value is about 3,960. Four: “one in four trips overflow to Lyft” is really a share of riders: 24.5 percent of riders rode Lyft. These corrections shaped everything after.

TRANSITION

With aggregate data only, we narrowed the question.

PREPARATION (not spoken)

Corrections (observed values from the digitized April aggregates).

1. **“About 5,100 rides per hour at 6 pm”** is the 30-day total for the 6 pm hour: tooltip 5,098, digitized 5,089. That is about 170 per evening.
2. **“About 28 vans”** is the “All services” average, 27.6 at 6 pm over 29 days.
3. **The 7 pm bar** reads 4,900 on the proposal chart, against a digitized 3,957 (`obs_p09`). We do not know how the proposal value was read. The 6 pm and 8 pm bars agree with our digitization to within 1%.
4. **“~1 in 4 trips overflow to Lyft”** is 24.5% of 44,374 riders (van plus Lyft). It is a share of riders, not of trips or requests: a Lyft trip can carry several riders, and many requests never become trips.

Also superseded (backup B9): the proposal’s “20-min pickup target (25 when busy)” is the wording of the live page read on 21 Oct. Pages captured around April 2026 said 15 / 20 / 20 (D-49).

Backup slides: slide 38 (B9)

Methodology PDF: §1.4 From the September proposal to the current study, p. 8; §3.1 Volume: how many requests, p. 12

Slide 5 · We narrowed the question to one controllable decision

Irfan Setiadi about 54 s · open slide 5

Why it is here. States the pivot as a scope choice, not as a claim about what synthetic data forbid.

SAY (exact narration)

We now hold one specified van dispatcher fixed and change only the Lyft transfer rule. This lets us study how the timing and information used by that rule change service and Lyft use. It is the question we chose; we don't know the internals of Hopkins' real dispatcher; and a controlled comparison needs everything else held fixed. Our van dispatcher re-plans every two minutes and at each new request, cancellation or no-show while waiting, or van coming on duty, modelled on the vendor TransLoc's published design from 2016. That is our modelling choice, not verified current Hopkins practice.

HANDOFF: Irfan Setiadi → Miles Zhou

That gives us the question. Miles will now explain how April's aggregate data became the ingredients of our scenarios.

PREPARATION (not spoken)

Design choice and its justification (technical review B).

- **We hold one specified dispatcher fixed** so that differences between runs come from the transfer rule.
- **Synthetic nights do not forbid comparing dispatch algorithms.** The same objection, that results depend on our assumptions, applies to rule comparisons too. Our scope reflects three things: the question we chose, the unknown internals of Hopkins' dispatcher, and the need for a controlled comparison.
- **Replanning cadence.** TransLoc's 2016 vendor post describes re-solving at each new ride, cancellation, driver-reported no-show and every 2 minutes. Our van dispatcher is modelled on that design (D-25). Its triggers differ slightly: a van coming on duty triggers a decision moment, while Lyft events do not. The current TransLoc version and any Hopkins dispatcher overrides are unknown (TransLoc notes §3.2).
- **The dispatcher is provisional** (D-51), pending a five-van speed check.

Likely questions.

- **Why not compare dispatch algorithms, as proposed?** We could, but it would be a different question. We chose the transfer decision because it is one controllable lever, and holding the dispatcher fixed isolates it.
- **Is your dispatcher Hopkins' dispatcher?** No. It is modelled on TransLoc's published 2016 design; Hopkins' current practice is not verified.

Script's own transition (alternative wording): So what exactly do April's dashboards contain?

Backup slides: slide 38 (B9)

> **Methodology PDF:** §1.4 From the September proposal to the current study, p. 8; §5.6 The dispatchers and the held-fixed choice, p. 31

Slide 6 · April describes totals—not individual journeys

Miles Zhou about 58 s · open slide 6

Why it is here. The evidence boundary that forces every later assumption.

SAY (exact narration)

We know how many requests ended in each status, where pickups and drop-offs accumulated, and how activity varied by hour. We cannot reconstruct each rider's journey. [optional] On the left is what April gives; on the right, what it never gives: when a rider asked, where they went, how long they waited, and what happened next. Throughout the talk we label inputs. Observed means read from the dashboards. Derived means computed from observed numbers. Assumed means our choice, to be tested; designed means an experiment setting we chose. We therefore built plausible evenings from the evidence, while keeping the missing pieces visible.

TRANSITION

The first piece is how busy an evening should be.

PREPARATION (not spoken)

Evidence. Digitized aggregates in `data/processed/obs_*`. The source file is restricted, so the deck shows no screenshots.

- Charts were digitized pixel by pixel and cross-checked against tooltips, for example 5,089 digitized against a 5,098 tooltip.

Why it matters. With no row-level link between request time, place, wait and outcome, every evening must be constructed, and every missing link becomes a stated assumption.

Backup slides: slide 33 (B4)

Methodology PDF: §2.1 What the dashboard gives, p. 10; §2.2 Why aggregates cannot pin down one scenario, p. 11

Slide 7 · How busy should a constructed evening be?

Miles Zhou about 64 s · open slide 7

Why it is here. Volume, derived step by step, with its assumptions in plain view.

SAY (exact narration)

We estimated a workload from April's completed rides, request-to-completion ratio and reported vehicle availability. About 5,100 completed rides at six o'clock over thirty days is about 170 in one evening. April had about 2.09 requests for every completed ride, so roughly 355 requests in that hour. Divided by 27.6 vehicles on duty, that is about 12.85 requests per vehicle per hour. This assumes the monthly request-to-completion ratio applies within the hour, and uses vehicle availability reported across services. It is an estimated workload, not observed arrivals. Our planned evening uses five vans, a team design choice; each hour then gets its own count, 64, 50 and 54: 168 requests. That five-van evening is planned, not yet run.

TRANSITION

What we actually ran is smaller, and deliberately harder.

PREPARATION (not spoken)

Derivation.

1. $C_{18} \approx 5{,}089$ over 30 days, so ≈ 170 completed rides per evening.
2. R/C = 50,297 / 24,095 = 2.087, so ≈ 355 requests in that hour.
3. $\rho_{18} = 170 \times 2.087 \div 27.6 \approx 12.85$ requests per vehicle-hour. The same formula gives $\rho_{19} = 10.10$ and $\rho_{20} = 10.71$.
4. For five vans: 5ρ_h = 64.25 / 50.50 / 53.55. Largest-remainder rounding to a total of 168 gives **64 / 50 / 54** (D-35, D-36).

Assumptions, stated on the slide.

- The monthly request-to-completion ratio holds within the hour.
- The vehicle count is reported across services.

The range. About 150–260 per three-hour evening; illustrative, not a confidence interval (D-49).

- About 10% rebookings → about 151.
- 20–35% fewer Homewood vehicles than "All services" → about 210–258.

Status. The five-van evening is **planned — not yet run**.

Likely questions.

- **Why per vehicle?** We simulate a small fleet, not all of April's service.
- **Why does hour 3 (54) exceed hour 2 (50)?** Fewer vehicles were on duty at 8 pm (24.6 against 27.3).

> **Backup slides:** slide 33 (B4)
>
> **Methodology PDF:** §3.1 Volume: how many requests, p. 12

Slide 8 · We started with a smaller, deliberately demanding hour

Miles Zhou about 56 s · open slide 8

Why it is here. Says exactly which experiment produced the results.

SAY (exact narration)

The current results come from two vans and 28 requests in one hour, repeated across 30 constructed nights. Each van has fourteen seats. Requests arrive between six and seven, and eleven of the 28 are placed in a burst between 6:20 and 6:30. Each of the 30 nights is different, and each is run under six transfer rules, defined shortly. This is a controlled pilot, not a reconstruction of a typical April night. Its load, fourteen requests per van-hour, is close to the 12.85 we estimated per vehicle across all services, but the burst concentrates demand on purpose, to stress the transfer decision.

TRANSITION

Next, where those requests go.

PREPARATION (not spoken)

Evidence (frozen inputs in `data/scenarios/v2/L5`, read by `make_numbers.py`).

- 30 nights; 28 requests each, all arriving 18:00–19:00.
- The generator places round(0.4 × 28) = 11 requests in 18:20–18:30, plus uniform background requests.
- Realized per night: 11–17 requests in that window (mean 13.7) and 34–51 passengers.
- 2 vans of 14 seats; van travel × 1.2.
- The spec's "$\geq$ 40%" is slightly off: 11/28 = 39%.

Why heavy. 14 requests per van-hour against an estimated 12.85, plus a burst, so that transfer decisions matter. Under vans only, about 16 of 28 requests are picked up.

Status. Tested in simulation.

Likely questions.

- **Why two vans, not five?** In the level ladder this is the dress rehearsal (spec §8), "too small for conclusions" by design. The five-van evening needs solve-time checks first (D-51); a pilot run already took up to 253 s.
- **Is 14 seats realistic?** It is the team's fleet decision (D-30). Seats rarely bind with parties of 1–5.

Backup slides: slide 33 (B4), slide 35 (B6)

Methodology PDF: §7.7 M3: the L5 pilot and the 30-night batch, p. 40; §4.12 Designed checks: the level ladder, p. 25

Slide 9 · Knowing the endpoints does not identify the journeys

Miles Zhou about 49 s · open slide 9

Why it is here. Geography: the maps give totals per place, never pairs.

SAY (exact narration)

Two different sets of trips can produce exactly the same pickup and drop-off totals. In this toy example, both versions have identical green totals, which are all the maps show, yet different trips. So pairing pickups with drop-offs is a modelling choice. We assumed nearby journeys were more likely, then balanced the synthetic trips to preserve the observed location totals. The rate at which distance lowers a pair's weight is our assumption; the data cannot identify it. [optional] The map shows one constructed hour built this way.

TRANSITION

The next input is who travels together.

PREPARATION (not spoken)

Method (D-18).

1. **Gravity prior:** $w_{od} = O_o \cdot D_d \cdot e^{(-\beta t_{od})}$, where t_{od} is driving minutes and $\beta = 0.1$ per minute, so the weight falls by about ×0.905 per minute. Pairs under 60 s of driving are excluded.
2. **Iterative proportional fitting:** rescale rows and columns in turn until both April location totals match. The largest error is about 10^{-10}.

Assumptions.

- β is a choice: every β reproduces the totals, so the data cannot choose it.
- Requests share the geography of completed rides.
- Times are uniform within an hour.
- The same geography holds every hour.

Not yet tested. A spread-out geography variant exists but has not been run.

Likely question.

- **Why prefer nearby trips?** It is the standard transport-planning prior. The alternatives (β = 0, or 0.2) are sensitivity options.

Backup slides: slide 33 (B4)

Methodology PDF: §3.3 Geography, p. 14; §3.4 Origin–destination pairing, p. 15

Slide 10 · One average can hide very different groups

Miles Zhou about 61 s · open slide 10

Why it is here. Party sizes affect seats and passenger-weighted waiting.

SAY (exact narration)

April helps identify an average group size. It does not tell us the full distribution of groups. April gives about 1.39 van passengers per completed van request. Examples A and B have exactly that average, yet one has eight percent of requests with three or more riders and the other over twelve. That matters for seats in the van, and because our waiting penalty counts every passenger. Our mix covers all requests, so its mean is slightly higher. Our chosen group-size mix is an assumption informed by the average—not a distribution recovered from April. A party of five that is moved to Lyft rides together in one larger XL vehicle.

TRANSITION

The hardest label on the dashboard is “Cancelled”.

PREPARATION (not spoken)

Derivation.

- **April:** 33,394 van passengers ÷ 24,095 completed van requests = **1.386**. This covers completed van requests only, so it is derived from two observed totals.
- **Example A** (0.72, 0.20, 0.054, 0.026, 0) and **Example B** (0.807, 0.07, 0.073, 0.03, 0.02) both have mean 1.386. Their 3+ shares are 8.0% and 12.3%. Both are illustrations, not data.
- **Ours** (D-38): (0.73, 0.18, 0.055, 0.025, 0.01), with mean 1.405 and 3+ share 9%. It describes all requests, so its mean need not equal April’s.

Why groups matter.

- **Seats.**
- **Waiting is counted per passenger:** the waiting penalty is party size × φ(wait), so a party of five waiting is five times as costly.
- **Lyft XL:** a transferred party of five rides together in one XL. This is a team decision (D-33), not data.

Likely question.

- **Why not fit the distribution?** One mean cannot identify five probabilities.

Backup slides: slide 33 (B4)

Methodology PDF: §3.5 Parties: how many riders per booking, p. 15

Slide 11 · One status label hides different events

Miles Zhou about 62 s · open slide 11

Why it is here. Separates an observation, an assumed status mapping and hidden patience.

SAY (exact narration)

‘Cancelled’ does not tell us whether a rider gave up or a dispatcher moved the request to Lyft. Almost half of April’s requests ended “Canceled”. We assume that requests moved to Lyft also close as “Canceled” in the van system, and we keep the share that is riders giving up as a range to test, fifty to seventy-five percent. To let riders give up in the simulator, each rider gets a private deadline the dispatcher cannot see, with a median of eighteen minutes. The patience distribution is provisional. It lets the simulator represent abandonment; it is not a behavioural distribution measured from April. [optional] Many patience distributions could produce the same cancellation share.

TRANSITION

A similar puzzle hides in “No Show”.

PREPARATION (not spoken)

Three layers.

- **Observed:** 23,843 of 50,297 (47.4%) ended “Canceled”. The digitized sums also show 99 “Denied”.
- **Assumed status mapping:** Lyft hand-offs close as “Canceled” (methodology Appendix C). Not confirmed.
- **Range to test:** the share that is rider abandonment, 50–75% (D-39, D-49). Reconstructed from the Lyft rider share and assumed party sizes; not established by the data.
- **Hidden patience:** a Weibull distribution with shape 1.5 and median 18 min, so scale = 18 / (ln 2)^(1/1.5) ≈ 23.0 min. Shape 1.5 means giving up becomes likelier the longer someone has waited.

Caution. A cancellation share does not uniquely determine a patience distribution: many shape and scale pairs give the same share.

Likely question.

- **Why 18 minutes?** It is a placeholder (D-39) until calibration. Results on slides 22–23 depend on it.

Backup slides: slide 33 (B4)

Methodology PDF: §3.7 Patience: when a waiting rider gives up, p. 16

Slide 12 · The simulator needs a probability at vehicle arrival

Miles Zhou about 55 s · open slide 12

Why it is here. Same count, two denominators; the simulator needs the per-arrival one.

SAY (exact narration)

The share of all requests marked ‘No Show’ is different from the chance of a no-show when a vehicle actually arrives. 2,359 no-shows out of all 50,297 requests is 4.7 percent. But a no-show can only happen when a vehicle arrives, and most cancelled requests never saw one. Out of van arrivals, completed rides plus no-shows, it is about 8.9 percent. That figure holds only if “No Show” was recorded after a real van arrival, “Completed” counts van trips, and Lyft hand-offs sit in “Canceled”. Our arrival-based probability is provisional because it depends on how the dashboard statuses were recorded.

TRANSITION

Together, these pieces form a scenario.

PREPARATION (not spoken)

Derivation.

- 2,359 / 50,297 = 4.7% of all requests.
- 2,359 / (24,095 + 2,359) = 2,359 / 26,454 ≈ 8.9% per van arrival.
- The arithmetic is exact; the uncertainty is entirely in the status mapping (D-48). The three conditions are on the slide.

Use.

- One presence draw per rider, used for whichever vehicle arrives, van or Lyft.
- The same probability for every mode is an assumption (D-48).

Likely questions.

- **Your scenario files say 4.7%?** A known erratum (D-48). Those fields hold all-request shares, which the simulator does not read; 8.9% is set in the run configuration.
- **Is a no-show related to how long the rider waited?** Not in our model; the data cannot identify such a link.

Backup slides: slide 33 (B4)

Methodology PDF: §3.8 No-shows: who is at the curb, p. 17

Slide 13 · The evidence becomes a scenario blueprint

Miles Zhou about 62 s · open slide 13

Why it is here. Gathers every input, with its status, before the model appears.

SAY (exact narration)

A scenario combines evidence-based inputs, explicit assumptions and hidden rider behaviour. Demand: arrival times, locations, trip pairing and party sizes. Resources: vans, their fourteen seats and travel times, which we inflate by twenty percent as a placeholder. Hidden behaviour: patience, presence at the curb and Lyft delay; a transfer takes two minutes to order, then the Lyft takes about eight minutes typically. And experiment settings: the van dispatcher, held fixed, and the transfer rule, which we vary. Many inputs carry two labels, derived and assumed. One point matters for what follows: inputs are fixed before a run, but cancellations, no-shows and transfers are not. They are outcomes of the run, not counts we impose.

HANDOFF: Miles Zhou → Youzhi Zhang

Those inputs define the constructed evening. Youzhi will explain how the model makes decisions as that evening unfolds.

PREPARATION (not spoken)

Combined labels. Many inputs are “DERIVED + ASSUMED”.

- Arrival times: the volume is derived; uniform timing and the burst are assumed or designed.
- Presence: derived conditionally on an assumed mapping.

Outcomes are not inputs. Cancellations, no-shows and transfers are produced by the run, from hidden draws meeting the rule and the vans. That is why paired runs of different rules can differ.

Inputs are frozen and hash-checked before any run. Backup B4 lists the provenance of each input.

Script’s own transition (alternative wording): Now, the machinery that runs a scenario.

Backup slides: slide 33 (B4)

Methodology PDF: §3.10 Patterns, seeds and pairing, p. 19; §1.2 Three things kept apart, p. 6

Slide 14 · Three components have different jobs

Youzhi Zhang about 65 s · open slide 14

Why it is here. The full architecture, with the information boundary visible.

SAY (exact narration)

Three components have different jobs. The simulator advances time and reveals events. The van optimizer chooses among feasible candidate plans. The transfer rule decides whether an eligible request moves to Lyft. [optional] The simulator holds the clock, the vans and every request's state; it reveals a request only when it arrives, and applies a hidden draw only when its event happens. The decision step, on the right, sees only what has happened: the van dispatcher generates candidate routes, an optimizer—a mixed-integer linear program—picks one per van, and the transfer rule reads those plans. Then the vans execute, and every event is recorded. Future requests, each rider's patience and presence, and the real Lyft delay stay hidden from the dispatcher.

TRANSITION

These parts work together in one repeating cycle.

PREPARATION (not spoken)

Mapping to code.

- Simulator: `src/sim/engine.py`.
- Route generation and optimizer: `dispatch.py`, a restricted route-selection MILP solved by HiGHS.
- Transfer rule: `lyft.py`.
- Scenario generator: `src/scenario/generator_v2.py`.

The information boundary. The dispatcher observes:

- arrived requests;
- van positions and riders aboard;
- committed stops;
- travel times.

Hidden draws (patience, presence, Lyft delay) are applied only by the simulator, when their events happen. So predictions and realized events can differ.

Likely question.

- **Does the rule see the hidden traits?** No. It reads only the current plan's predicted pickups, or a tentative estimate.

Backup slides: slide 32 (B3)

Methodology PDF: §1.2 Three things kept apart, p. 6; §Fig. 1 The system in one picture, p. 7; §4 The simulator contract in plain language, p. 19

Slide 15 · The same decision cycle repeats as the evening unfolds

Youzhi Zhang about 58 s · open slide 15

Why it is here. The verified implementation order of one decision moment.

SAY (exact narration)

The dispatcher sees requests that have arrived. It does not see future requests. Every two minutes, and at each new request, waiting-time cancellation or no-show, or van coming on duty, the same cycle runs. Reveal and update. Generate feasible routes for each van. Select one route per van. Evaluate the transfer rule for each request still waiting. Apply accepted transfers: those requests leave van planning, and the Lyft is ordered two minutes later. If anything was transferred, re-plan once. Then vans execute and commit to their next stop. The rule runs once per decision moment, so there is no cascade of transfers in the same second.

TRANSITION

How does the optimizer pick a route?

PREPARATION (not spoken)

Verified implementation order (`engine.py:305–351`; technical review).

1. Same-second events are processed first, in fixed classes; patience checks come before the decision moment.
2. The solver generates candidates and the MILP picks one route per van.
3. The unassigned timer is updated (and again after any re-solve).
4. The rule reads each request still waiting for a van.
5. Declines are recorded. Transfers leave van planning, with the Lyft ordered at now + 2 min.
6. If at least one transfer happened, there is **one** re-solve, with no second rule pass.
7. Departures; a van commits to its next stop.

Triggers in the code.

- A new request.
- A cancellation or no-show while waiting for a van.
- A van coming on duty.
- The 120-s tick.

A van going off duty and Lyft events do not trigger a decision moment. The spec lists off-duty as a trigger, a small spec/code difference recorded in the technical review. At most one decision moment runs per second.

Likely question.

- **Could transfers cascade?** No. One rule pass and at most one re-plan per decision moment.

Backup slides: slide 32 (B3)

> **Methodology PDF:** §4.3 Same-second events, p. 21; §4.4 Epochs: when the dispatcher re-plans, p. 21; §4.5 The Lyft rule pass, p. 21

Slide 16 · The optimizer compares the consequences of candidate plans

Youzhi Zhang about 76 s · open slide 16

Why it is here. The objective in blocks, one worked example, and what “optimal” means.

SAY (exact narration)

Waiting, riding, driving and unresolved requests enter the model in different ways. Waiting is weighted by party size and is convex: each extra minute costs more the longer someone has waited. Riding, at half a point per passenger-minute, and driving, at a tenth per kilometre, always count less than waiting. A request given to no van in this round is postponed; postponing has a price, plus a large constant, so the plan first serves as many requests as its candidates allow. Seats and pickup-before-drop-off are hard constraints; ride time has a softer guardrail. In this example, picking up the rider who has waited eight minutes first costs 66.5 points; the other order costs 81. [optional] That rider would pass fifteen minutes. ‘Optimal’ means best among the candidate plans generated at that decision moment—not the best possible whole evening.

TRANSITION

The van optimizer stays fixed. What we change is the transfer rule.

PREPARATION (not spoken)

Objective. $c = \Sigma\, q \cdot \varphi(W) + 0.5\, \Sigma\, q \cdot H + 0.1$ km (+ Ψ only for an unavoidable existing guardrail excess). φ is piecewise linear:

Minutes waited	**0–5**	**5–10**	**10–15**	**15–20**	**20+**
Slope (points per passenger-minute)	1	2.5	5	10	20

- So $\varphi(5) = 5$, $\varphi(10) = 17.5$, $\varphi(15) = 42.5$ and $\varphi(20) = 92.5$.
- The 16th minute of waiting costs 10 points.
- The riding weight 0.5 is below every waiting slope.

Worked example (dwell omitted).

- **Route A:** W = 12 and 8, H = 8 and 9. Waiting 27.5 + 2×12.5 = 52.5; riding 0.5×(8+18) = 13.0; driving 1.0. Total **66.5**.
- **Route B:** W = 6 (party) and 16; riding 0.5×(20+5) = 12.5. Total **81.0**.

Postponement. $\pi_r = b_r + M$. M guarantees that the fewest requests the candidates allow are postponed (B2). It does not guarantee that every request is served.

Constraints.

- **Hard:** seats; precedence; one route per van; each request served once or postponed.
- **Guardrail:** max(25, 2×direct + 5) min. No plan may create or worsen an excess, but a no-show wait can still push a ride over: 36 events, 16 distinct requests (B8).

"Optimal." HiGHS proves optimality over the candidates generated at that decision moment: every subset of at most 3 new pickups per van, each in its best stop order. It is not optimal over all routes, nor over the evening.

Likely questions.

- **Why at most 3 new pickups?** Exact ordering with 4 took minutes per decision in Python.
- **Optimal, given 41 incomplete tie searches?** The main solve was proven optimal at all 13,663 decision moments. Enumeration showed the optimum unique in all 41.

Backup slides: slide 31 (B2), slide 37 (B8)

Methodology PDF: §5.2 What a "good" plan means, p. 26; §5.3 The route-selection model, p. 28; §5.4 Worked example, p. 29; §5.5 What "optimal" means here, p. 31

Slide 17 · We changed the transfer decision, not the van dispatcher

Youzhi Zhang about 69 s · open slide 17

Why it is here. Defines all six rules from the code, before any result.

SAY (exact narration)

We tested six rules, one at a time. Vans only never transfers. Wait ten minutes and Wait fifteen minutes move a request once it has waited that long. Unassigned for ten minutes moves a request left out of every van plan for ten minutes in a row. Predicted total wait exceeds fifteen minutes uses the van plan: predicted total wait equals predicted pickup clock time minus request clock time. And Lyft arrives sooner fires if the predicted van pickup is later than now plus the two-minute ordering lag, an eight-minute typical Lyft delay and a two-minute margin. We call these last two the predictive rules; the first ones use only time. Only requests still waiting, not committed to a van, are eligible. The thresholds were specified before any run.

TRANSITION

Here is that last rule, worked through.

PREPARATION (not spoken)

Code definitions (`lyft.py:13–30`). τ = now, t_r = request time.

Rule	Moves the request when
Wait ten / fifteen minutes	$\tau - t_r \geq 600$ / 900 s
Unassigned for ten minutes	τ − unassigned_since $\geq$ 600 s
Predicted total wait exceeds fifteen minutes	predicted pickup − t_r > 900 s
Lyft arrives sooner	predicted pickup > τ + 120 + 480 + 120 s, i.e. now + 12 min

Predicted pickup. The plan's ETA if the request is assigned, otherwise the tentative ETA. If there is no feasible van, it is infinite and the rule fires.

Eligibility. State "waiting for a van": not committed to a van, never having declined.

Reset example for "Unassigned for ten minutes" (preparation only).

- A request arrives at 18:00 and is outside every plan. Its timer starts at 18:00.
- At 18:06 it enters a van's plan: the timer resets.
- At 18:08 a re-plan drops it again: the timer restarts at 18:08.
- It becomes eligible only at 18:18, if it stays unplanned that long, even though it has waited 18 minutes.

The timer is updated at decision moments, so "continuous" means at decision-moment granularity.

Thresholds.

- Fifteen minutes echoes the published pickup-target wording around April 2026 (wording, not verified operating policy; D-49).

> - Ten minutes is an earlier trigger.
> - Two minutes is a margin against prediction error.
>
> All six were specified before any run (spec §6–§7) and not changed after seeing results. Pre-registration v2 is still a draft, to be frozen before the five-van experiment, so do not say “pre-registered”.
>
> **Backup slides:** slide 30 (B1)
>
> **Methodology PDF:** §4.5 The Lyft rule pass, p. 21; §6.2 The transfer path, p. 34

Slide 18 · What does ‘Lyft arrives sooner’ actually compare?

Youzhi Zhang about 64 s · open slide 18

Why it is here. One verified case, on a small designed evening.

SAY (exact narration)

The rule compares predicted van pickup with expected Lyft pickup, including the ordering delay. On a small designed evening, a request arrives at about 6:16. At 6:17 the van plan predicts pickup at 6:30 and eight seconds. A Lyft ordered then is expected at 6:27: two minutes to place the order plus a typical eight-minute delay. That is three minutes eight seconds sooner, more than the two-minute margin, so the request moves. The Lyft is ordered at 6:19 and reaches the curb at 6:27, ten and a half minutes after the request. The van pickup time is a prediction. We do not observe what that van would have done after choosing a different action.

TRANSITION

Testing at this level of detail also exposed a weakness.

PREPARATION (not spoken)

Evidence. `outputs/sim/L4_R3–2`: the epoch at 1,020 s, and request R011 in `outcomes.csv`. Checked by `make_numbers.py`, which asserts that the rule fires.

Step	Time
Request	18:16:30
Decision	18:17:00
Van plan’s predicted pickup	18:30:08
Expected Lyft = 18:17:00 + 2 + 8 min	18:27:00 (gap 3 min 8 s > 2 min)
Lyft ordered	18:19:00
Lyft at the curb (realized)	18:27:00
Dropped off	18:30:25
Wait, request to curb	10 min 30 s

Expected vs realized. On this designed evening the Lyft delay is fixed at 8 min, so the realized arrival equals the expected one by design. Acceptance is 0.9, and this rider accepts.

Correction from v0.3. The drop-off at 18:30:25 is **17 seconds after** the predicted van pickup at 18:30:08, not before it.

Likely question.

- **Did the transfer help this rider?** We cannot say. The van time is a prediction, and the run never shows what the van would have done. Whole evenings are compared instead (slide 20).

Backup slides: none specific

Methodology PDF: §4.6 Transfers and the Lyft service, p. 22; §6.2 The transfer path, p. 34

Slide 19 · A cheap postponement price could keep a rider waiting

Youzhi Zhang about 60 s · open slide 19

Why it is here. A flaw found by testing, compared, and only partly fixed.

SAY (exact narration)

The first formulation could price postponement more cheaply than a physically possible pickup. At 6:15 on the designed evening, a party of five had waited fifteen minutes. Postponing them was priced at 262.5 points, but the earliest pickup any van could actually make was worth about 436. We compared three versions over that whole evening and adopted the one that removes the flaw without adding variables: price postponement at the earliest time any van could physically arrive. The adopted approximation reduced the worst wait in this designed case. It did not reduce average waiting. And it is still optimistic, because it ignores the nearest van's other work.

TRANSITION

With the model settled, how do we compare rules fairly?

PREPARATION (not spoken)

The flaw. The original price $b_r = q \cdot \varphi(\text{next tick} - t_r)$ assumes a van at the next 2-min tick. At 18:15 a party of five that had waited 15 min was charged 262.5 points, against 435.8 for the earliest physical pickup at 18:19:28.

Controlled comparison: the designed evening, vans only, one run per formulation (B5).

	Median / 90th pct / worst wait (min)	Mean (min)	Extra variables
First version	17.1 / 31.0 / 44.4	18.8	0
Adopted	18.6 / 31.2 / 35.6	18.8	0
Route-based	21.1 / 34.3 / 38.6	21.8	up to 9,828

Adopted, provisionally (D-53). $b_r = q \cdot \varphi(\max(\text{next tick}, L_r) - t_r)$, with L_r the earliest any on-duty van can reach the pickup.

- It changes cost coefficients only.
- Computing L_r is $|\text{open}| \times |\text{vans}|$ lookups per decision, which was not separately timed. "No added variables" is not a runtime measurement.

Limitations.

- The adopted price is still optimistic when the nearest van is busy, and it does not guarantee oldest-first service.
- On this evening the youngest request waited 35.6 min instead of 9.8, so the median rose.
- The route-based version can err in either direction.

Likely questions.

- **Your fix made the median worse?** Slightly. We adopted it because the old price was provably below any service a van could give, not for outcomes on one evening. The 90th percentile also rose slightly (31.0 → 31.2 min).
- **Who paid for the lower worst wait?** One request that had waited little went from 9.8 to 35.6 minutes. The worst case moved; it did not vanish.
- **Why "the one that removes the flaw" rather than the simplest?** The first version is simpler, but it carries the flaw. The adopted one fixes the flaw with no added variables; the route-based one adds up to 9,828.

Backup slides: slide 34 (B5)

Methodology PDF: §7.4 A property of the objective that M1 exposed, p. 37; §7.5 The pending-price decision experiment, p. 37

Slide 20 · Replay the same night; change one rule

Youzhi Zhang about 44 s · open slide 20

Why it is here. The experimental design behind every result.

SAY (exact narration)

Each rule faces the same requests and hidden rider inputs, so differences reflect the rule within this specified model. We replay each constructed night six times, with the same requests, hidden traits, Lyft delays and vans, changing only the rule. Thirty nights replayed six ways are thirty paired scenarios—not 180 independent nights. Pairing removes night-to-night differences from each comparison; it does not make the nights representative of April. The settings are provisional, including the placeholder patience.

TRANSITION

Before trusting any result, we checked the implementation.

PREPARATION (not spoken)

Pairing. Per night, the requests, hidden patience, presence, Lyft delay per rider and the vans are identical across all six rules (D-57). 180 runs are 30 paired scenarios.

Pilot settings (D-56).

- Ordering lag 2 min.
- Acceptance 1.0.
- Presence p = 0.089.
- Frozen placeholder patience.
- Van travel × 1.2.

Likely question.

- **Are 30 nights enough?** For the main gap, descriptively yes: both predictive rules reach more than vans only on 30 of 30 paired nights (B7). For the waiting-time rules and “Unassigned”, no. Intervals wait on the agreed criteria (D-50); the draft pre-registration sets a sample-size rule (60 nights per cell if the expected half-width exceeds 0.7 δ).

Backup slides: slide 36 (B7)

Methodology PDF: §3.10 Patterns, seeds and pairing, p. 19; §7.7 M3: the L5 pilot and the 30-night batch, p. 40

Slide 21 · We checked implementation before interpreting outcomes

Youzhi Zhang about 54 s · open slide 21

Why it is here. Verification versus validation, and who wrote the code.

SAY (exact narration)

Verification asks whether the code implements the specified model. Validation asks whether the model represents the real service. On verification: three small designed evenings reproduce hand-computed event times to the second. A separate brute-force program checked about 290,000 plan combinations on a designed evening and always agreed with the optimizer. 253 automated tests pass, and the reruns we sampled were byte-identical. We should be clear: AI wrote the simulator code and the checks, under our direction. Separate implementations are useful checks, but they share our assumptions and do not replace our own understanding. Validation against real riders is still ahead.

HANDOFF: Youzhi Zhang → Vignesh Rajesh Babu

We then replayed the same evenings under different transfer rules. Vignesh will explain what changed, what it means and what remains uncertain.

PREPARATION (not spoken)

What each claim covers.

- **"To the second":** L1–L3 designed evenings match hand-computed times for every van, curb arrival, pickup, drop-off, cancellation and no-show.
- **On L4:** the five transferring rules match the hand key, with 22 transfers (4 XL) and 4 declined offers in total over five runs.
- **"292,181":** a separate brute-force program enumerated every compatible combination of candidate routes on the designed evening: 151,710 under the first price and 140,471 under the adopted one. It always found the MILP's choice. This checks the optimizer **over the same candidates**, not global optimality.
- **"253 tests":** 46 cover scenario generation and 207 the simulator; 168 were written by separate AI review agents that did not reuse the simulator's code.
- **"6 of 6":** six sampled reruns were byte-identical. This is a sample, not every run.

Who produced the hand keys? The keys sit with each designed evening (`data/scenarios/v2/L1/HANDCHECK.md` and `handcheck.csv`, likewise for L2–L4). The engine's events are compared with them by two independent test files (methodology §7). Check `docs/ai_use_log.md` for whether the keys were also prepared with AI assistance. Before claiming human verification, the team should confirm which keys a team member re-derived by hand, and say so honestly if none were.

Who wrote what. The simulator and the checks were written with an AI coding assistant under the team's direction, logged in `docs/ai_use_log.md`. Separate implementations share our specification and assumptions. They catch coding errors, not modelling errors, and they do not replace our own understanding.

Likely question.

- **Who wrote the code and checks?** Answer first and plainly, as above.

Script's own transition (alternative wording): Now, what the rules did.

Backup slides: slide 37 (B8)

Methodology PDF: §7.2 What was verified, p. 36; §7.7 M3 verification, p. 40

Slide 22 · The predictive rules reached more requests promptly—and used more Lyft

Vignesh Rajesh Babu about 64 s · open slide 22

Why it is here. The main result, with its conditions in the subtitle.

SAY (exact narration)

These are thirty constructed burst-demand nights with provisional rider behaviour. Each bar is the share of all requests a vehicle reached at the curb within fifteen minutes—curb arrival, not boarding or a completed trip. In this two-van pilot, the predictive rules reached about three quarters of requests within fifteen minutes, compared with about half under vans only. They also used roughly nine to ten and a half Lyft orders per night. The waiting-time rules added little, partly because a rider moved after ten minutes almost never counts within fifteen; on requests picked up, the order is the same. This comparison does not isolate the benefit of prediction at an equal Lyft budget.

TRANSITION

But reaching a rider with a Lyft is not the end of the story.

PREPARATION (not spoken)

Results (means over 30 nights; SD in brackets).

Rule	Reached $\leq$ 15 min	Lyft orders / night
Vans only	0.477 (0.095)	0.00
Wait ten minutes	0.514 (0.082)	5.93
Wait fifteen minutes	0.495 (0.090)	3.10
Unassigned for ten minutes	0.476 (0.096)	0.23
Predicted total wait exceeds fifteen minutes	0.735 (0.074)	9.33
Lyft arrives sooner	0.750 (0.075)	10.47

The metric is curb arrival within 15 min over all 28 requests. Wait percentiles cover only requests picked up, so they drop those who cancelled.

The metric favours early transfer. A request moved under "Wait ten minutes" has waited at least 10 min, then faces the 2-min ordering lag and a Lyft delay with median 8 min. It reaches the curb within 15 min only if the delay is under about 3 min, roughly 2%. Under "Wait fifteen minutes", never. So the waiting-time rules can gain on this measure only by freeing vans. Measures with no threshold give the same ordering: requests picked up, 20.2–20.4 against 16.1–17.2, and the 90th-percentile wait (B6). Sensitivity at 10 and 20 min is not yet run.

Lyft orders vs transfers. A transfer cancelled during the ordering lag never becomes an order: 196 − 18 = 178 = 5.93 × 30.

What the comparison does not isolate.

- The predictive rules also used more Lyft, so their advantage is not measured at an equal Lyft budget.
- Per Lyft order relative to vans only, they add about 0.73–0.77 requests reached, against about 0.16–0.17 for the waiting-time rules.
- That ratio is descriptive and does not control for budget.

Likely questions.

- **Isn't your metric rigged for early transfer?** Partly, as above. The ordering is the same on measures with no threshold.
- **Your predictive rules transfer at 0 s elapsed. Isn't that just sending the burst straight to Lyft?** Largely, yes: they act at the first decision moment at which the plan already predicts a long wait. They differ from the waiting-time rules in *when* they move a rider, not in whether they do.
- **Do the pilot nights resemble April?** Only loosely. Under vans only, the 90th-percentile wait among requests picked up is 20.2 min, against April's 17–21 min for completed rides. That is face plausibility, not validation, and our percentiles exclude cancellations.
- **Is the prediction rule better?** It reaches more requests and uses more Lyft. "Better" needs agreed criteria (D-50).
- **Why did "Unassigned for ten minutes" do almost nothing?** It was identical to vans only on 24 of 30 nights, with 8 transfers on 6 nights, all of requests made during the burst (moved at 18:35–18:38). Requests were usually in some van's plan within minutes even when pickup was far off: assignment is not pickup.

Backup slides: slide 35 (B6)

Methodology PDF: §7.7 M3: the L5 pilot (mechanism 5: the 15-minute measure), p. 41

Slide 23 · Moving a request to Lyft does not end its wait

Vignesh Rajesh Babu about 58 s · open slide 23

Why it is here. What happens after transfer, stated no more strongly than the evidence.

SAY (exact narration)

The rider still waits through ordering and Lyft arrival. Their original patience clock continues running. Under Wait ten minutes, 95 of 196 transferred requests cancelled before their Lyft arrived, some before it was even ordered; under the prediction rules, about a third. Riders moved by the prediction rules had usually waited less when they were moved. With our placeholder patience, where giving up becomes likelier the longer someone has waited, that is consistent with their lower share lost after transfer. It is a possible contributing mechanism, not an isolated cause, and whether the gap survives other patience assumptions is untested.

TRANSITION

So rules trade service against Lyft use. How do we compare them?

PREPARATION (not spoken)

Fates of transferred requests (totals over 30 nights).

Rule	Transfers	Lyft completed	Lyft no-show	Cancelled in lag	Cancelled after order	Median wait already elapsed at transfer
Wait ten minutes	196	94	7	18	77	629 s
Wait fifteen minutes	104	55	4	11	34	932 s
Unassigned for ten minutes	8	4	0	1	3	698 s
Predicted total wait exceeds fifteen minutes	294	187	14	14	79	0 s
Lyft arrives sooner	330	211	15	16	88	0 s

Supported interpretation (technical review A).

- A transfer does not end waiting; riders may still cancel before Lyft arrives.

- Riders moved by the prediction rules had usually waited less. Under the placeholder, where giving up becomes likelier with time waited, that is consistent with their lower loss: a conditional calculation on those riders gives 44–47% against 29–30%, matching what was observed.
- Under constant-hazard patience, elapsed waiting alone does not change the probability of surviving an equal additional waiting period. That does not establish that the cancellation gap between entire policies disappears: different patience would change who is transferred, when, and who survives in the vans.
- Whether the gap survives other patience shapes is **untested**.
- "Wait fifteen minutes" loses fewer than "Wait ten minutes" despite later transfers, so selection matters too.

Likely question.

- **Isn't this just your patience assumption?** That a transferred rider keeps waiting until the Lyft reaches the curb is built into the model. The size of the loss, and the gap between rules, depend on the placeholder. Testing that at policy level is the first proposed next step.
- **Per night, are fewer riders lost after transfer under the prediction rules?** Not fewer per night: about 3.1–3.5 against 3.2 (B6). They transfer more riders, so their *share* lost is lower.

Backup slides: slide 35 (B6)

Methodology PDF: §7.7 M3: the L5 pilot (mechanism 1 and its interpretation), p. 41; §3.9 Lyft: what happens after a transfer, p. 18

Slide 24 · Better service and lower Lyft use can point to different choices

Vignesh Rajesh Babu about 66 s · open slide 24

Why it is here. Pareto dominance, used descriptively.

SAY (exact narration)

A tested rule is dominated if another tested rule is at least as good on both selected objectives and strictly better on one. In the small picture, A dominates B; A and C do not dominate each other. On the thirty-night means, five of our six rules are not dominated. Vans only stays nondominated simply because it uses no Lyft. Unassigned for ten minutes differs from vans only by about one request in 840, so it is effectively a tie that uses slightly more Lyft; we apply no noise margin yet. Nondominated does not mean recommended. This is the nondominated set among six tested rules under these two objectives—not the complete frontier of possible policies. [optional] Adding objectives or criteria could change the assessment.

TRANSITION

Which raises a natural question.

PREPARATION (not spoken)

Definitions.

- **Service:** S = share reached $\leq$ 15 min (higher is better).
- **Lyft use:** B = Lyft orders per night (lower is better).
- **Dominance:** P dominates Q if $S_P \geq S_Q$ and $B_P \leq B_Q$, with at least one strict.
- **Ties:** identical points do not dominate each other.

Results.

- **Five of six are nondominated on the 30-night means.** No noise margin is applied.
- **"Unassigned for ten minutes" is dominated by vans only:** 0.476 against 0.477 service, and 0.23 against 0 Lyft orders. It is identical to vans only on 24 nights; on the other 6 it reached more on 2 and fewer on 2. No statistical margin is applied (D-50).
- **Vans only is nondominated through zero Lyft use alone.**

Supported and unsupported points. Minimising $-S + \lambda B$ for any $\lambda \geq 0$ picks only vans only, "Predicted total wait…" or "Lyft arrives sooner". The two waiting-time rules lie below the line from vans only to "Predicted total wait…" (0.477 + 0.0276 B), so no weighting selects them.

Night by night (B7). The prediction rules reach more than vans only on all 30 nights; "Wait ten minutes" on 19, and fewer on 6.

Three trade-offs kept separate.

1. The routing weights inside the optimizer.

2. This policy comparison.
3. The decision criteria, not yet agreed.

Likely question.

- **Is the Pareto set robust to noise?** Not established. Per night, the predictive rules are nondominated on 24 and 12 of 30 nights (B7).
- **Why Lyft orders, not cost?** No Lyft cost is known (D-45). Orders count a regular Lyft and an XL equally, so a cost axis could change the dominance.
- **Is this the Pareto frontier?** No: it is the nondominated set among six tested rules. A frontier would come from threshold sweeps, and a frontier over all policies from the extension on slide 25.

Backup slides: slide 36 (B7)

Methodology PDF: §7.8 Reading the results as trade-offs, p. 42

Slide 25 · Could we choose the transfer policy systematically?

Vignesh Rajesh Babu about 66 s · open slide 25

Why it is here. Leader–follower as a proposed extension, stated precisely.

SAY (exact narration)

One possible extension is to optimize transfer policy while anticipating how the routing dispatcher responds. A policy designer chooses the rule and threshold; the dispatcher responds by re-planning; the outcomes inform the choice. This is proposed, not implemented. It would be bilevel because the dispatcher optimizes its own routing points, which differ from the designer's service and Lyft criteria. A finite search does find the best option in a finite set for the average over our sampled nights, but not over all policies or every continuous threshold. [optional] To state it precisely, we would still need to define the leader's choices, the follower's problem and timing, how ties are treated, the expectation over nights, and the weight on Lyft use.

TRANSITION

Before choosing anything, we need to know which assumptions matter.

PREPARATION (not spoken)

Corrected framing (technical review C).

- **A threshold need not be a leader–follower problem.** It can be tuned by simulation-based search, with the dispatcher as a black box; a bilevel formulation is one modelling choice.
- **A finite search can solve a finite problem.** If the follower is defined as our deterministic per-decision restricted MILP with its tie rule, exhaustive search over a finite threshold set finds the best option for the defined objective, averaged over the sampled nights. It does not give optimality over continuous thresholds or all policies.

One precise version. min over θ of E[L(θ, y) + λ·B(θ)], subject to y being the follower's response. To make it precise, define:

1. **The leader's space:** rule family, a discrete or continuous threshold, bounds.
2. **The follower:** a sequence of per-decision restricted MILPs whose state depends on earlier decisions and random outcomes, or one whole-evening problem.
3. **Non-unique follower optima:** optimistic or pessimistic (Colson, Marcotte & Savard 2007, *Ann. Oper. Res.* 153:235–256; Dempe 2002). Ours uses a fixed canonical tie rule (D-52), which is neither.
4. **The expectation:** a scenario distribution, or a sample average over N nights.
5. **The loss L and the weight λ** (D-50 open).
6. **Upper-level constraints**, such as a Lyft budget or a service floor.

Likely questions.

- **Isn't your loop already leader–follower?** No. The rule reads the current plan, then the optimizer re-plans once. Nothing chooses the rule by anticipating the response.

- **Riders and Lyft as players?** Not modelled as optimizing.
- **Why bilevel and not just simulation optimisation?** It becomes bilevel only if we model the dispatcher as optimizing its own routing points, which differ from the designer's service and Lyft criteria. Simulation-based search treating the dispatcher as a black box is a valid alternative.
- **Is your follower optimistic or pessimistic?** Neither. A fixed canonical tie rule (D-52) makes its response unique. In 41 cases the search for equally good plans was incomplete; in all 41 the optimum was shown unique (B8).

Backup slides: slide 31 (B2)

Methodology PDF: §7.9 A leader–follower reading, p. 43

Slide 26 · Which assumptions could change the answer?

Vignesh Rajesh Babu about 54 s · open slide 26

Why it is here. Limits ranked by how they could change the interpretation.

SAY (exact narration)

The findings are conditional on the evenings and behaviours we constructed. Patience is the assumption we most expect to matter, though we have not tested that: it sets how many riders give up and the losses after transfer. Lyft delay: a slower or more variable Lyft could weaken transfers; under our placeholder patience, late transfers most. Workload: the pilot is deliberately heavy; at lighter load the rules may barely differ. Geography changes van reach and predictions. [optional] And the candidate restriction, three new pickups per van, shapes both van performance and the predictions the rules read.

TRANSITION

That ordering tells us what to test next.

PREPARATION (not spoken)

Priority and reasons.

- **Patience** is the assumption we most expect to matter (untested): it drives both abandonment and the post-transfer losses.
- **Lyft delay:** a slower or more variable Lyft could weaken transfers; under our placeholder patience, late transfers most.
- **The 15-minute measure** favours early transfer (slide 22).
- **The workload and burst** were chosen to stress the rules.
- **Geography** changes van reach and the predictions the rules read.
- **The candidate restriction** shapes both van performance and those predictions.

Also open.

- The 8.9% no-show rate depends on the status mapping.
- A two-van pilot, not five vans.
- Verification, not validation.
- 36 ride-time excess events (16 distinct requests on 10 nights), each coinciding with a no-show wait on that van: attribution, not proof.

Backup slides: slide 37 (B8)

Methodology PDF: §7.11 Limitations to keep in view, p. 44; §2.3 Calibration keeps a set, not a point, p. 11

Slide 27 · Test robustness before fine-tuning a winner

Vignesh Rajesh Babu about 40 s · open slide 27

Why it is here. Proposed next steps, in order.

SAY (exact narration)

Next, we should test whether the findings survive different rider-patience and Lyft-delay assumptions, while checking feasibility at the larger fleet size. Then the team must agree how much extra Lyft use one more rider reached is worth, compute confidence intervals across the paired nights, and only then search over thresholds. Order matters: a threshold tuned to placeholder behaviour would be tuned to our assumptions, not to the service.

TRANSITION

So, what can we defend today?

PREPARATION (not spoken)

Order, and why.

1. **Behavioural sensitivity:** patience shape and median, Lyft delay. This is the policy-level test the slide 23 interpretation leaves open.
2. **Five-van feasibility:** solve times and the candidate cap (D-51).
3. **Agreed criteria:** margins and the value of Lyft use (D-50), then paired intervals on saved nights.
4. **Threshold or policy search:** only after 1–3.

These are proposals; nothing here authorizes running them.

Backup slides: slide 37 (B8)

Methodology PDF: §7.12 Next milestone and how the work grows, p. 45; §7.13 Open decisions for the team, p. 46

Slide 28 · What we can defend today

Vignesh Rajesh Babu about 28 s · open slide 28

Why it is here. The conditional answer; no universal recommendation.

SAY (exact narration)

Aggregate data did not give us individual journeys. It gave us constraints for building plausible evenings. Within our specified pilot, predictive transfer rules improved timely curb arrival while using more Lyft. Choosing a policy requires testing the assumptions and deciding what trade-off is acceptable. Thank you; we welcome your questions.

TRANSITION

(End of the talk; questions follow.)

PREPARATION (not spoken)

What the closing sentence rests on.

- Slides 6–13: aggregates gave constraints, not journeys.
- Slides 20–22: the conditional pilot result.
- Slides 23–27: why choosing a policy needs assumption tests and agreed criteria.

Likely question.

- **So which rule should Hopkins use?** We do not recommend one. Within our two-van pilot, prediction rules reached more requests promptly at the cost of more Lyft. Choosing requires robustness tests and a judgement about Lyft use.

Backup slides: slide 37 (B8)

Methodology PDF: §7.10 Status by item, p. 44